



Instituto Tecnológico de Costa Rica
Sede San Carlos

Sistema de Control de Préstamos

Programación Orientada a Objetos

Kristel Paola Munoz Quesada

Tabla de Contenidos

1. Introducción.....	3
2. Descripción del problema.....	4
2.1 ¿Qué se busca resolver?.....	4
2.2 Subproblemas identificados.....	4
3. Metodología.....	5
3.1 Solución de los subproblemas.....	5
3.2 Explicación del diagrama de clases.....	5
3.3 Tecnologías utilizadas.....	6
4. Análisis crítico.....	7
6. Conclusiones.....	8
7. Recomendaciones.....	8

1. Introducción

El presente documento describe el proceso de diseño e implementación del sistema de Control de Préstamos desarrollado para el proyecto del curso de POO. El objetivo principal de este proyecto fue construir una aplicación utilizando Java que permita administrar préstamos de diferentes tipos de ítems, aplicando principios fundamentales de la programación orientada a objetos y patrones de diseño.

Durante el desarrollo del sistema se implementaron diferentes módulos independientes para la administración de categorías, personas, tipos, ítems y préstamos. Cada módulo fue implementado con una función específica dentro del sistema para el control y mantenimiento del problema.

Además del desarrollo de las funcionalidades, el proyecto permitió aplicar conceptos vistos en el curso, como encapsulamiento, patrones de diseño, relaciones entre clases, excepciones e interfaces gráficas aplicando el uso de Java Swing.

Este documento presenta el problema a resolver con el desarrollo del sistema, la metodología utilizada para resolverlo, las herramientas utilizadas y un análisis crítico de los resultados.

2. Descripción del problema

2.1 ¿Qué se busca resolver?

Muchas instituciones necesitan controlar el préstamo de diferentes recursos, como computadoras, proyectores, herramientas entre otros objetos comunes. Cuando este proceso se hace manualmente se tienen a perder el control del tiempo y personas que las poseen, complicando la recuperación y control de las mismas para la empresa.

El objetivo del proyecto fue desarrollar una aplicación que permite administrar toda esta información de forma centralizada, facilitando el registro de las personas y las pertenencias, la organización de los recursos y el seguimiento en cada préstamo.

Para lograrlo fue necesario diseñar una estructura orientada a objetos que representará correctamente las entidades y relaciones entre las partes existentes.

2.2 Subproblemas identificados

Durante el análisis del proyecto se identificaron varios subproblemas que debían resolverse para cumplir con el objetivo principal.

Administración de personas: Era necesario contar con un módulo que permita registrar a todas las personas que vayan a utilizar el sistema. Además del registro era necesario poder buscar a una personas, modificar su información, eliminar registros cuando fuera permitido y constar la lista completa de personas registradas.

Administración de categorías: Los ítems debían organizarse mediante categorías, esto facilita la búsqueda y clasificación. Esto requiere implementar un módulo independiente encargado de registrar y administrar dichas categorías.

Administración de tipos: Cada ítem pertenece a un tipo específico, para esto fue necesario desarrollar otro módulo que permita administrar los diferentes tipos de recursos utilizados por el sistema.

Administración de ítems: El sistema debía de permitir registrar todos los recursos disponibles para los préstamos, almacenando información como nombre, código, descripción, tipo. También debía de ser posible eliminar, modificar y editar dichos productos además de poder listarlos.

Gestión de préstamos: Este es el subproblema principal. El sistema debía de permitir asociar a una persona con uno o varios ítems, registrando toda la información del ítem, fecha del préstamo y mantener actualizado el estado de disponibilidad de cada recurso

Finalización de préstamos: Una vez devueltos los recursos, el sistema debía de poder finalizar el préstamo adecuadamente, liberando nuevamente el recurso para un

préstamos futuro para otra o la misma persona, manteniendo información básica del préstamo registrada.

3. Metodología

En esta sección se describe la metodología utilizada durante el desarrollo del Sistema de Control de Préstamos, detallando cómo se abordó cada uno de los subproblemas de la etapa de análisis. También se explica la estructura del sistema a partir de bibliotecas patrones de diseño, colecciones y bibliotecas implementadas para construir la aplicación funcional como solución al problema de préstamos.

3.1 Solución de los subproblemas

Para desarrollar el sistema se decidió dividir el problema principal en varios módulos independientes. Esta forma de trabajo permitió implementar y probar cada funcionalidad de manera individual antes de integrarla con el resto del sistema.

El desarrollo inició con la construcción del modelo del sistema a partir del diagrama de clases, definiendo las entidades principales como Persona, Categoría, Tipo, Ítem y Préstamo. Posteriormente se implementó la clase **ControlPréstamos**, encargada de centralizar toda la lógica del negocio y administrar la información almacenada.

Una vez implementada la lógica principal, se desarrolló la interfaz gráfica utilizando Java Swing. Cada módulo cuenta con ventanas independientes para registrar, buscar, modificar y eliminar información, facilitando la interacción del usuario con el sistema.

Para garantizar la integridad de los datos se implementaron validaciones antes de realizar cada operación. Por ejemplo, el sistema verifica que no existan personas duplicadas, que los identificadores ingresados sean válidos y que un ítem no pueda ser prestado si ya se encuentra asociado a un préstamo activo. Estas validaciones se realizaron mediante excepciones personalizadas, permitiendo mostrar mensajes claros al usuario cuando ocurre alguna situación inválida.

Finalmente se realizaron pruebas funcionales sobre cada módulo implementado con el objetivo de comprobar el correcto funcionamiento de las operaciones de registro, búsqueda, modificación, eliminación y gestión de préstamos.

3.2 Explicación del diagrama de clases

La idea central es que una persona puede realizar varios préstamos, y cada préstamo pertenece a una persona específica. Por eso la clase Persona guarda sus datos principales, como nombre, identificación, teléfono y correo, también mantiene una lista de préstamos asociados.

- La clase Préstamo es la que representa cada préstamo realizado dentro del sistema, por eso tiene identificador, fecha de préstamo, fecha de retorno, estado, una persona relacionada, una posible alerta y una lista de ítems prestados.

- La clase Ítem representa los objetos que se pueden prestar. Cada ítem tiene un código, nombre, descripción, estado de préstamo y está relacionado con un tipo, un préstamo y una o varias categorías. Por eso existe la relación con Tipo, porque cada ítem pertenece a un tipo en específico como por ejemplo computadora, equipo de laboratorio. También se relaciona con Categoría ya que un ítem puede pertenecer a varias categorías y una categoría puede contener varios ítems.
- La clase Categoría permite agrupar o clasificar los ítems, mientras que la clase Tipo define una clasificación más directa del ítem. En el caso de Tipo, también se agregó un atributo genérico porque en el sistema se maneja un tipo por defecto cuando sea necesario y tanto Categoría como Tipo mantienen listas de ítems, porque desde el código se permite agregar o eliminar ítems relacionados.
- La clase Alerta está asociada con préstamo y sirve para manejar recordatorios o avisos relacionados con devolución. Por eso tiene atributos como mensaje, fecha de inicio, frecuencia y si está activada o no. También se agregó el método debeActivarse(), que fue parte de las observaciones que me diste, así permite que la propia alerta determine si debe activarse según el tiempo transcurrido y la frecuencia configurada. La frecuencia se maneja mediante el enum FrecuenciaAlerta, donde se definen valores como UNA_VEZ, DIARIA, SEMANAL y MENSUAL.
- La clase ControlPrestamos funciona como la controladora principal del sistema. Esta clase administra las colecciones de personas, ítems, categorías, tipos, préstamos y alertas mediante estructuras Map, porque permite buscar rápidamente los objetos por su identificador o en el caso de persona por su identificación. También contiene métodos para crear, modificar, eliminar, listar y consultar los datos del sistema, y métodos específicos para agregar ítems a un préstamo, retornar ítems, finalizar préstamos, asignar alertas y generar reportes.
- También agregué un paquete de Errores, donde se colocaron las excepciones propias del sistema, se usan para manejar situaciones específicas por ejemplo cuando una persona no existe, un ítem ya prestado, un préstamo está finalizado o una categoría no se encuentra. Todas estas clases heredan de Exception, y la controladora las utiliza para lanzar errores más claros en lugar de solo devolver un false así los errores son más fáciles de identificar. Así resumidamente las relaciones principales son una persona realiza préstamos, un préstamo contiene ítems, un préstamo puede tener una alerta, un ítem pertenece a un tipo y un ítem puede estar en varias categorías.

Colecciones

La información del sistema se almacena mediante estructuras HashMap, lo que permite acceder rápido a cada objeto utilizando su identificador como llave.

Además, se utilizaron ArrayList para representar relaciones entre objetos, por ejemplo la lista de préstamos asociados a una persona y los ítems pertenecientes a cada préstamo.

Patrón de diseño

El patrón de diseño utilizado fue Singleton, implementado en la clase ControlPrestamos.

Este patrón garantiza que exista una única instancia del controlador durante toda la ejecución del programa.

Algoritmos implementados

Las principales operaciones se implementaron usando algoritmos de registro, búsqueda, eliminación y modificación de información.

En el módulo de préstamos se implementó un proceso que verifica la existencia de la persona y la disponibilidad de los ítems antes de registrar el préstamo. Una vez validado se realiza la operación y el estado de los recursos se actualiza automáticamente.

Asimismo, al finalizar un préstamo, el sistema actualiza nuevamente el estado de cada ítem para dejarlo disponible para futuros préstamos.

4. Análisis crítico

El desarrollo de este proyecto representó un reto importante, ya que fue necesario aplicar la mayoría de los temas vistos durante el curso para construir una aplicación funcional. A lo largo del proceso fue necesario analizar el problema antes de comenzar a programar, con el fin de definir correctamente las clases, sus relaciones y la forma en que cada módulo interactuaría con los demás.

Uno de los aspectos que más tiempo requirió fue la implementación de la lógica del sistema, especialmente en el módulo de préstamos, donde fue necesario realizar diferentes validaciones para evitar inconsistencias en la información. También fue un reto integrar la lógica con la interfaz gráfica, procurando que cada acción realizada por el usuario produjera el resultado esperado.

Considero que uno de los mayores logros del proyecto fue desarrollar un sistema organizado, donde cada módulo cumple una función específica y permite administrar personas, categorías, tipos, ítems y préstamos de una manera sencilla. Además, el uso de excepciones personalizadas y del patrón Singleton permitió mantener un mejor control sobre la información y hacer el código más ordenado.

Aunque el sistema cumple con los principales objetivos planteados, existen algunos aspectos que podrían mejorarse en una futura versión. Entre ellos se encuentra la implementación completa del módulo de reportes, así como la incorporación de una base de

datos para conservar la información de forma permanente y no depender únicamente de la memoria durante la ejecución del programa.

En general, este proyecto me permitió fortalecer mis conocimientos sobre programación orientada a objetos y comprender mejor la importancia de planificar el diseño antes de comenzar la implementación. También me ayudó a desarrollar una mejor organización del código y a comprender cómo se construye una aplicación de escritorio utilizando Java.

6. Conclusiones

El desarrollo de este proyecto permitió aplicar de forma práctica los conocimientos adquiridos durante el curso de Programación Orientada a Objetos. A través de la implementación del sistema fue posible comprender mejor la importancia de realizar un buen diseño antes de comenzar a programar, así como la utilidad de dividir el problema en módulos independientes.

Además, este proyecto me permitió reforzar el uso de clases, relaciones entre objetos, colecciones, excepciones personalizadas e interfaces gráficas en Java. Considero que el sistema desarrollado cumple con el objetivo principal de administrar préstamos de una manera organizada y facilitar el control de la información.

7. Recomendaciones

Como trabajo futuro, sería conveniente implementar el módulo de reportes para ampliar las funcionalidades del sistema y facilitar la consulta de la información.

También sería recomendable incorporar una base de datos que permita almacenar la información de forma permanente y mejorar la interfaz gráfica para hacerla más intuitiva y agradable para el usuario. Estas mejoras permitirían que el sistema sea más completo y pueda adaptarse a necesidades reales de una organización.